plishes nothing; a constrained file-management tool can perform the operation.

Think of tools as bridges between language and systems.

A useful first tool set might contain:

- `calculate()` for arithmetic.
- `search_files()` for local document lookup.
- `lookup_product()` for a SQLite database.
- `read_csv_summary()` for analysing a table.
- `get_order_status()` for an authorised business system.
- `create_report()` for writing a result to a designated output folder.

Avoid beginning with a tool named `run_any_command`. A narrowly scoped tool is easier to describe, validate, test and secure.

Tool quality also affects model quality. If two functions have vague names such as `search()` and `find()`, the model may choose inconsistently. Give each tool a precise name, a short description and unambiguous parameters. Current Transformers tooling can derive a tool schema from appropriately typed Python functions and their docstrings; Hugging Face recommends descriptive names, type hints and well-formed argument descriptions because this information is what the model uses to understand the available operation.

Your first tool call

We will build the simplest possible loop before introducing frameworks. Our example assistant can multiply two numbers. The arithmetic is intentionally trivial: we are testing orchestration, not showing off mathematics.

Create `src/tools_demo.py` and define the function:

```
def multiply(a: float, b: float) -> float:
    """Multiply two numbers."""
    return a * b

TOOLS = {
    "multiply": multiply
}
```

Now imagine the user asks, "What is 17.5 multiplied by 83?"

A tool-capable model should not return the result directly if you have instructed it to use the calculator. Conceptually, it emits something like:

```
{
  "name": "multiply",
  "arguments": {
    "a": 17.5,
    "b": 83
  }
}
```

Your program must then perform four steps.

Step 1: Parse the requested tool. Extract the function name and arguments using the tool-calling format expected by your chosen model. Do not scrape arbitrary prose with fragile regular expressions if the model/runtime offers structured tool-call parsing.

Step 2: Check the allowlist. Confirm that `multiply` exists in `TOOLS`. Never turn a model-produced string into `eval()` or an unrestricted shell command.

Step 3: Validate the arguments. Confirm that `a` and `b` are numbers and reject unexpected fields. A model can produce malformed arguments even when it usually behaves correctly.

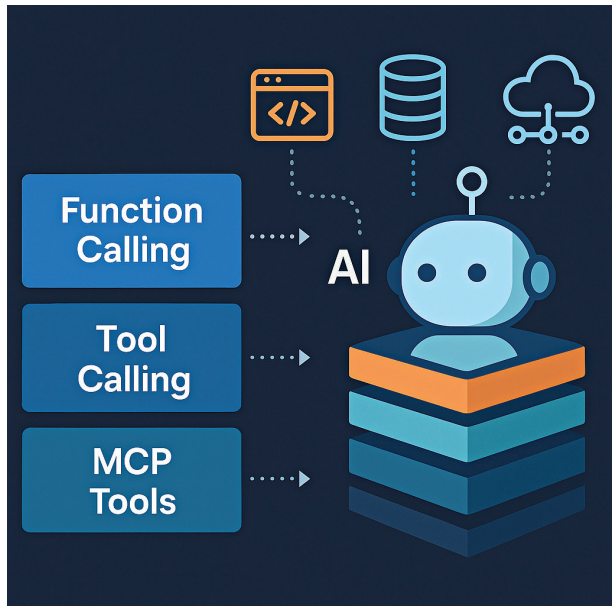
Step 4: Execute and return the result. Run `TOOLS["multiply"](**arguments)`, convert the result into the message format expected by the model, and add it to the conversation as a tool result. Then ask the model to continue.

The lifecycle looks like this:

User → model → tool request → validator → Python function → tool result → model → user

Transformers follows essentially this pattern. Tool definitions can be passed to compatible chat templates; when the model asks for one, the application appends the assistant's tool call, executes it and then appends a message with the `tool` role containing the result. The exact tokens still depend on how that model was trained, so reuse its chat template rather than inventing your own format.

This is the first major architectural shift in our project. The model proposes. Your application disposes.



Make outputs machine-readable

Tool use works best when information moves between components in predictable structures. Natural language is excellent for humans but awkward for software. Compare these two outputs:

"Looks like product A is in stock and there are twelve units."

and: